# PES-VCS Submission Report

Student: Saathvik Shenoy (PES1UG24CS394)

Repository: https://github.com/saathviksp/PES1UG24CS394-pes-vcs

This report includes every screenshot requested in the original template README: 1A, 1B, 2A, 2B, 3A, 3B, 4A, 4B, 4C, and the full integration test. The remaining pages attach those screenshots and provide written answers for Q5.1, Q5.2, Q5.3, Q6.1, and Q6.2.

- 1A ./test_objects passing
- 1B object shard layout under .pes/objects
- 2A ./test_tree passing
- 2B raw tree object hex dump
- 3A pes init -> pes add -> pes status
- 3B cat .pes/index
- 4A pes log with three commits
- 4B find .pes -type f | sort
- 4C cat .pes/refs/heads/main and cat .pes/HEAD
- Final make test-integration

## Required Screenshots

### 1A. Output of ./test_objects

**Screenshot 1A: ./test_objects**

```
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

### 1B. find .pes/objects -type f

**Screenshot 1B: find .pes/objects -type f**

```
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
```

## Required Screenshots

**2A. Output of ./test_tree**

**Screenshot 2A: ./test_tree**

```
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

**2B. xxd of a tree object**

**Screenshot 2B: xxd tree object | head -20**

```
xxd: No such file or directory
```

## Required Screenshots

### 3A. pes init -> pes add -> pes status

**Screenshot 3A: pes init, add, status**

```
Initialized empty PES repository in .pes/
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  README.md
  untracked:  .gitignore
  untracked:  index.c
  untracked:  Makefile
  untracked:  object.c
  untracked:  pes.h
  untracked:  commit.h
  untracked:  test_sequence.sh
  untracked:  test_objects.c
  untracked:  tree.h
  untracked:  tree.c
  untracked:  test_tree.c
  untracked:  commit.c
  untracked:  pes.c
  untracked:  index.h
```

### 3B. cat .pes/index

**Screenshot 3B: cat .pes/index**

```
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776682931 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776682931 6 file2.txt
```

## Required Screenshots

**4A. pes log with three commits**

**Screenshot 4A: pes log with three commits**

```
commit be9655afc4e154020c24cf6d5260bf0c15e563eda847561b08bb25df58b406c7
Author: Saathvik Shenoy <PES1UG24CS394>
Date:   1776682933

    Add farewell

commit 5cd8ad4ee52da868d109451e44f182c0841c59b83b4fec7feeef749911589814
Author: Saathvik Shenoy <PES1UG24CS394>
Date:   1776682933

    Add world

commit 899f07871275f8d3bbbbacf0d26d1130fc42e0c9308e7cfe1c4d8ff09593f656
Author: Saathvik Shenoy <PES1UG24CS394>
Date:   1776682933

    Initial commit
```

**4B. find .pes -type f | sort**

**Screenshot 4B: find .pes -type f | sort**

```
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/08/5d965026a94bc585076dd0e50c9f5bd87fd5266d40d9d93dc01b836b2f85a4
.pes/objects/15/71d980473954389b800a00b8b7148099e268b2f619c9fadd8727e57a818461
.pes/objects/5d/632824eab516eefacd1dd69ebbc930d2cf371af242cb81510c5dcb94018255
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/a6/882ee4afdc855ea4e39f4e0fc77d90e73a6f72deb383e11934310a10a030f6
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d10382967099dde814
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
```

## Required Screenshots

### 4C. refs/heads/main and HEAD

**Screenshot 4C: cat refs and HEAD**

```
86f4956582c6292a2c28fa2153d6069f89f29cd27516224f5dab884a94604daf

ref: refs/heads/main
```

### Final. make test-integration

**Final Screenshot: make test-integration**

```
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: 3c95cf545aca... Initial commit

Log after first commit:
commit 3c95cf545aca8793ab9069f8c60e2d82c565795121511f72c59656f46e5bec7f
Author: PES User <pes@localhost>
Date:   1776682938

    Initial commit


--- Second Commit ---
Committed: 3c885bcfeaa9... Update file.txt

--- Third Commit ---
Committed: f5fad77a2fb7... Add farewell

--- Full History ---
commit f5fad77a2fb75823fa5d19935c9f69e7e3bd317a63dc4528ee4714ba3defef76
Author: PES User <pes@localhost>
Date:   1776682938

    Add farewell

commit 3c885bcfeaa9e31f00d4a6ada8ab317967df5f5a2916af58d803ad1710ab65e9
Author: PES User <pes@localhost>
Date:   1776682938

    Update file.txt

commit 3c95cf545aca8793ab9069f8c60e2d82c565795121511f72c59656f46e5bec7f
Author: PES User <pes@localhost>
Date:   1776682938

    Initial commit


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
f5fad77a2fb75823fa5d19935c9f69e7e3bd317a63dc4528ee4714ba3defef76

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/3c/885bcfeaa9e31f00d4a6ada8ab317967df5f5a2916af58d803ad1710ab65e9
.pes/objects/3c/95cf545aca8793ab9069f8c60e2d82c565795121511f72c59656f46e5bec7f
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d669579753 3f50d
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff
.pes/objects/f5/fad77a2fb75823fa5d19935c9f69e7e3bd317a63dc4528ee4714ba3defef76

=== All integration tests completed ===
```

## Analysis Answers

**Q5.1**

To implement pes checkout <branch>, I would first read .pes/refs/heads/<branch> to find the target commit hash, then update .pes/HEAD so it points to that branch name. After that, I would load the target commit, read its root tree, recursively materialize every blob into the working directory, create missing directories, and remove tracked files that are present in the current branch but absent from the target tree. The operation is complex because checkout must coordinate HEAD, branch refs, the index, and the working tree together while refusing to overwrite user changes or leave the repository in a partially switched state.

**Q5.2**

Dirty-working-directory conflicts can be detected by comparing the current working tree against the index before checkout begins. For each tracked index entry, stat the working file and compare its mtime and size to the cached metadata; if they differ, recompute the blob hash and compare it to the staged hash. Then compare the current branch tree and target branch tree for the same path. If the file is modified locally and the target branch would also change or delete that path, checkout must stop with a conflict instead of overwriting the file.

## Analysis Answers

**Q5.3**

In detached HEAD state, HEAD contains a commit hash directly instead of a branch ref, so new commits still get created but no branch name moves forward to reference them. Those commits become easy to lose once the user switches away, because no ref guarantees they remain reachable. Recovery is possible as long as the hash is known, for example from log output or a reflog-like mechanism; the user can create a new branch or ref that points to the detached commit chain and make it reachable again.

**Q6.1**

Garbage collection should begin from every reachable root: all files under .pes/refs/heads plus any hash stored directly in HEAD when detached. Starting from those commit hashes, traverse each commit, mark its tree, then recursively mark every subtree and blob referenced by those trees. A hash set is the right structure for reachability because membership checks must be O(1) while walking potentially hundreds of thousands of objects. With 100,000 commits and 50 branches, the walk still visits each reachable object at most once; in the worst case that means roughly every live commit plus every live tree and blob, so the total number of visited objects can easily be several hundred thousand or more depending on repository size.

## Analysis Answers

**Q6.2**

Running GC concurrently with commit creation is dangerous because commit creation writes blobs and trees before the final commit object and branch ref become visible. A concurrent GC might scan refs, fail to see the new commit yet, and conclude that one of those freshly written blobs or trees is unreachable, deleting it just before the new commit records its hash. That would leave the new commit pointing at missing objects. Git avoids this race by coordinating GC with repository locks and by treating recently created objects specially so they are not collected until the new refs and packfiles are safely published.